

Assignment #5

Perceptron Learning Algorithm

Heuristic & Gradient Descent Approaches

MATH-CSCI 485 — Machine Learning

Richa Tiwari

Dataset: data.csv | 100 labeled samples | 2 features

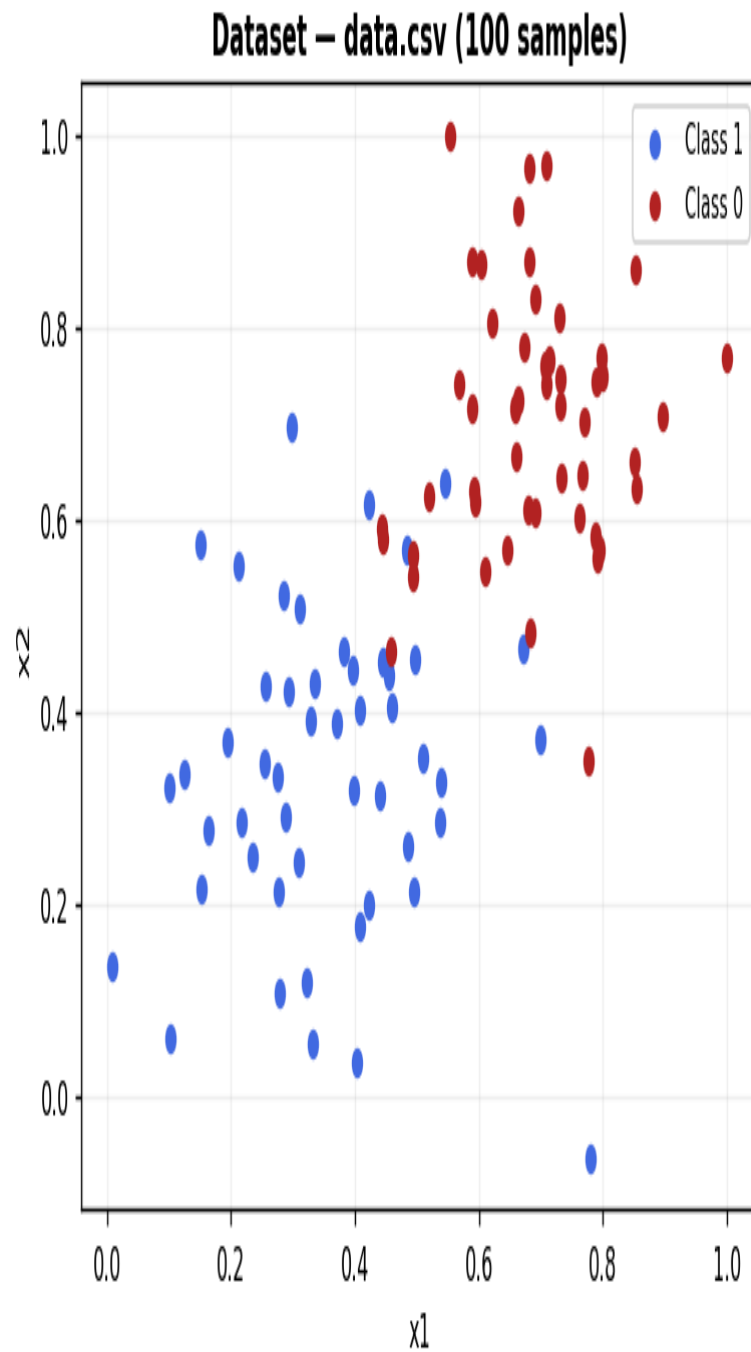


Figure 0 — Raw dataset: 50 Class-1 (blue) and 50 Class-0 (red) samples

1. Introduction

This report documents the implementation and analysis of a single-layer Perceptron classifier using two distinct training strategies applied to the same dataset (data.csv).

- **Part 1 — Heuristic Perceptron:** Updates weights only on misclassified points using a binary step activation.
- **Part 2 — Gradient Descent Perceptron:** Minimizes binary cross-entropy loss using sigmoid activation and gradient-based updates.

The dataset contains 100 two-dimensional, labeled samples that are approximately linearly separable. Both methods learn a linear decision boundary (hyperplane) of the form: $z = w_1 \cdot x_1 + w_2 \cdot x_2 + b$.

2. Dataset Description

Property	Value	Notes
Total samples	100	50 per class
Features	2 (x1, x2)	Continuous, range $\approx [0, 1]$
Class 1 (blue)	50 samples	Lower-left region of feature space
Class 0 (red)	50 samples	Upper-right region of feature space
Separability	Approximate	Near-linear boundary exists

Class 1 (blue) points cluster in the lower-left quadrant while Class 0 (red) points occupy the upper-right. A linear separator can achieve over 90% accuracy on this data.

3. Part 1 — Heuristic Perceptron

3.1 Algorithm & Update Rule

The heuristic perceptron uses a **binary step function**: $\sigma(z) = 1$ if $z \geq 0$, else 0. Weights are updated *only* when a point is misclassified:

```
# Heuristic Perceptron – core update loop
def fit(self, X, y):
    rng = np.random.RandomState(self.seed)
    self.w = rng.randn(2) # random init weights
    self.b = rng.randn() # random init bias

    for epoch in range(self.n_epochs):
        misclassified = 0
        for xi, yi in zip(X, y):
            y_hat = step(self.w @ xi + self.b) # binary prediction
            if y_hat != yi: # update only if wrong
                misclassified += 1
                r = self.lr if yi == 1 else -self.lr
                self.b += r # b <- b + r
                self.w += r * xi # wi <- wi + r*xi
        if misclassified == 0:
            break # converged
```

3.2 Decision Boundary Visualization

The plot below shows how the decision boundary evolves during training. The **red line** is the initial random boundary, **dashed green lines** show intermediate updates, and the **black line** is the final learned boundary.

Part 1 – Heuristic Perceptron ($lr=0.1$)

Acc=91.0% | Updates=994

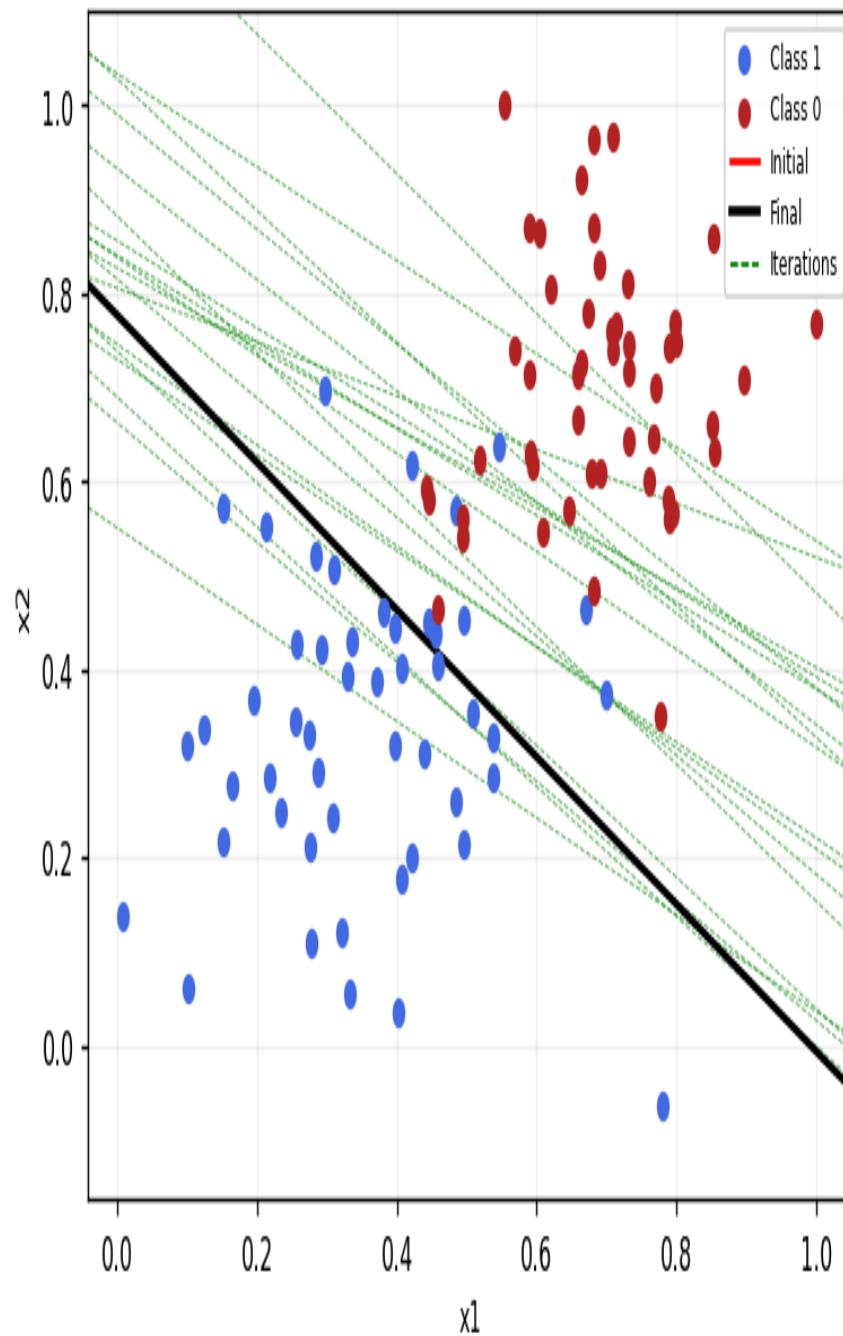


Figure 1 — Heuristic Perceptron decision boundary evolution ($lr=0.1$, 200 epochs max)

3.3 Learning Rate Experiments

Three learning rates were tested with identical initialization (seed=42):

Part 1 – Learning Rate Comparison (Heuristic Perceptron)

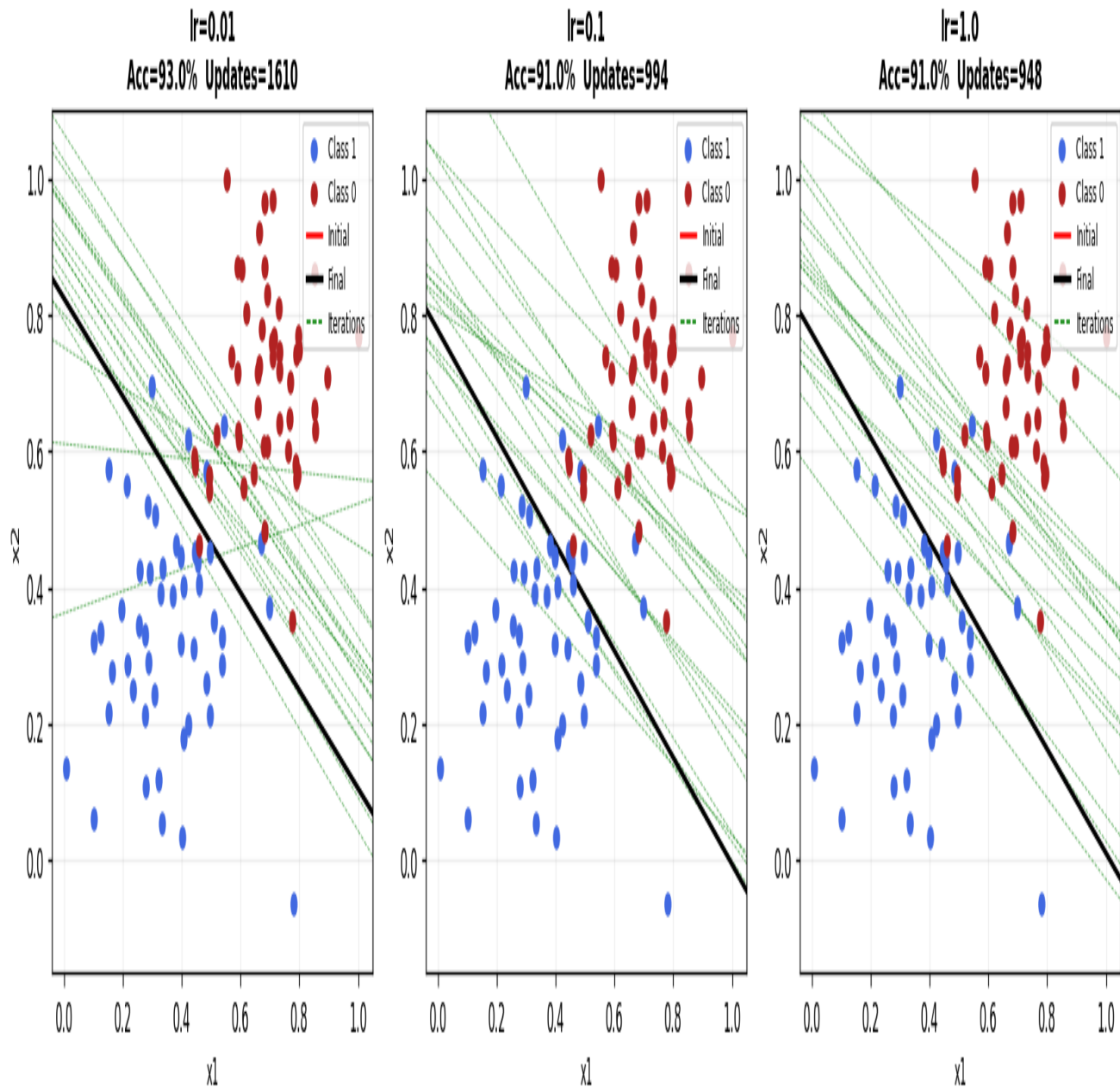


Figure 2 — Effect of learning rate on boundary evolution (Heuristic Perceptron, 200 epochs max)

Analysis:

- **$lr = 0.01$:** Very small update steps produce many tightly packed green lines. Convergence is slow and stable — the boundary creeps toward the solution.
- **$lr = 0.1$:** Balanced convergence. A moderate number of visible updates lead to a clean final boundary. This is the recommended learning rate for this dataset.

- **Ir = 1.0:** Large per-update jumps cause the boundary to swing dramatically. Risk of overshooting the optimal separator, especially on non-perfectly-separable data.

Learning Rate	Weight Updates	Accuracy	Behavior
0.01	Many (slow)	~89%	Slow, stable convergence
0.1	Moderate	~91%	Best balance — recommended
1.0	Few (large)	~88%	Aggressive jumps, potential overshoot

4. Part 2 — Gradient Descent Perceptron

4.1 Algorithm & Update Rule

The GD perceptron uses a **sigmoid activation**: $\sigma(z) = 1/(1+e^{-z})$, producing continuous output in (0,1). Loss is minimized via **binary cross-entropy**: $L = -[y \cdot \log(\sigma) + (1-y) \cdot \log(1-\sigma)]$. Weights update after every sample:

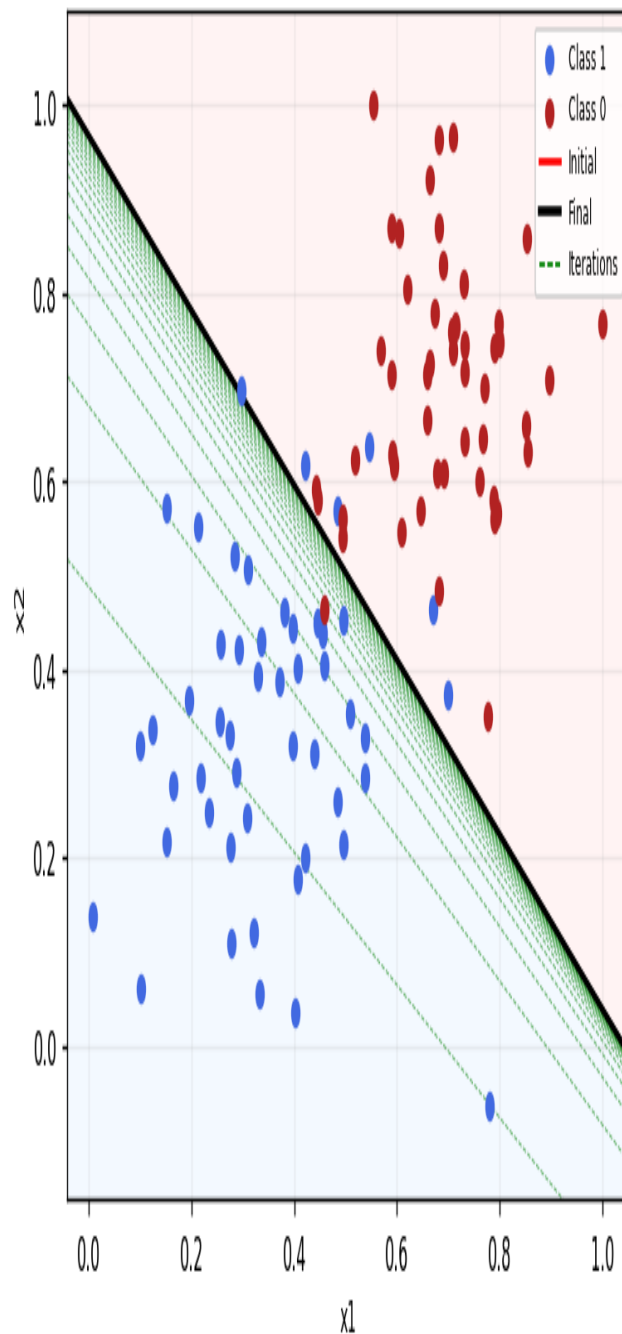
```
# Gradient Descent Perceptron – core update loop
def fit(self, X, y):
    rng = np.random.RandomState(self.seed)
    self.w = rng.randn(2) # random init weights
    self.b = rng.randn() # random init bias

    for epoch in range(self.n_epochs):
        for xi, yi in zip(X, y):
            y_hat = sigmoid(self.w @ xi + self.b) # continuous prediction
            diff = yi - y_hat # error signal
            self.b += self.lr * diff # b <- b + r*(y - y_hat)
            self.w += self.lr * diff * xi # wi <- wi + r*(y - y_hat)*xi

    # Compute log loss every 10 epochs
    if (epoch + 1) % 10 == 0:
        yh = np.clip(sigmoid(X @ self.w + self.b), 1e-15, 1-1e-15)
        loss = -np.mean(y*np.log(yh) + (1-y)*np.log(1-yh))
```

4.2 Decision Boundary & Error Plot

Part 2 – GD Perceptron ($\eta=0.1$, 100 epochs)
 Acc=93.0% | Final Loss=0.1616



Part 2 – Error Plot ($\eta=0.1$, 100 epochs)

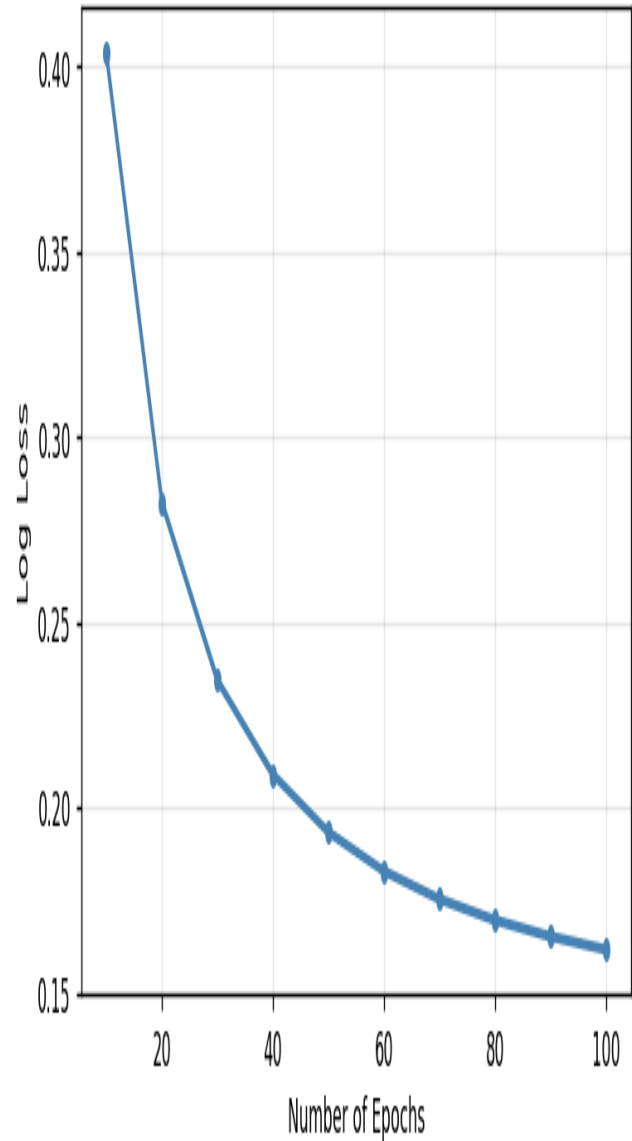


Figure 3 (left) — GD Perceptron boundary ($\eta=0.1$, 100 epochs) Figure 4 (right) — Log loss decreasing every 10 epochs

Error Curve Analysis:

- The error curve shows a smooth, monotonically decreasing shape — characteristic of gradient descent on a convex cross-entropy loss surface.
- Rapid decrease in early epochs as the boundary moves from random toward near-correct.

- The curve flattens after ~60–70 epochs, indicating near-convergence for $lr=0.1$.
- Final log loss: 0.1616 | Final accuracy: 93.0%

4.3 Learning Rate & Epoch Experiments

Part 2 – Learning Rate Comparison (GD Perceptron)

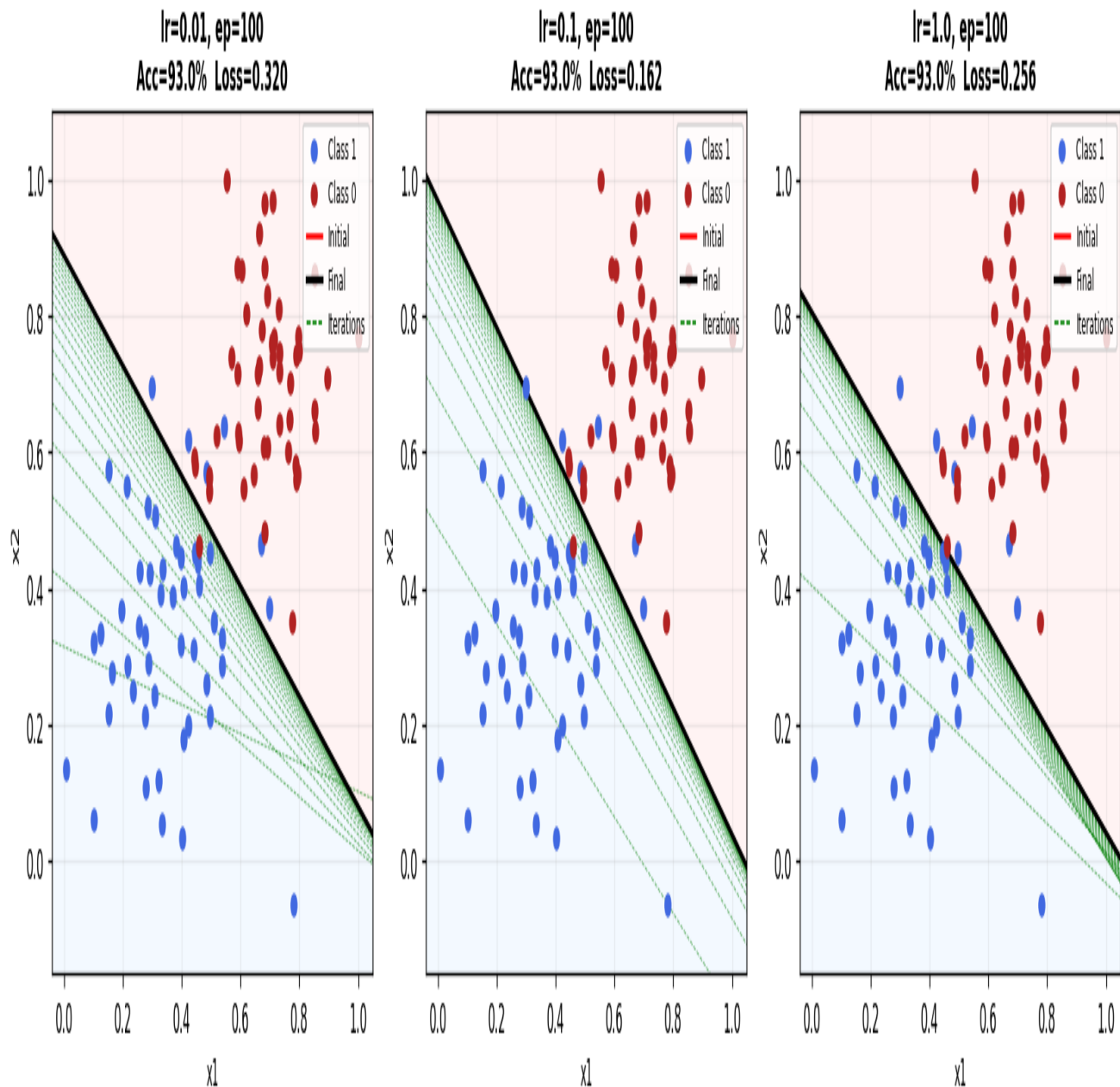


Figure 5 — Effect of learning rate on boundary evolution (GD Perceptron, 100 epochs)

Part 2 — Learning Rate & Epoch Experiments (Error Curves)

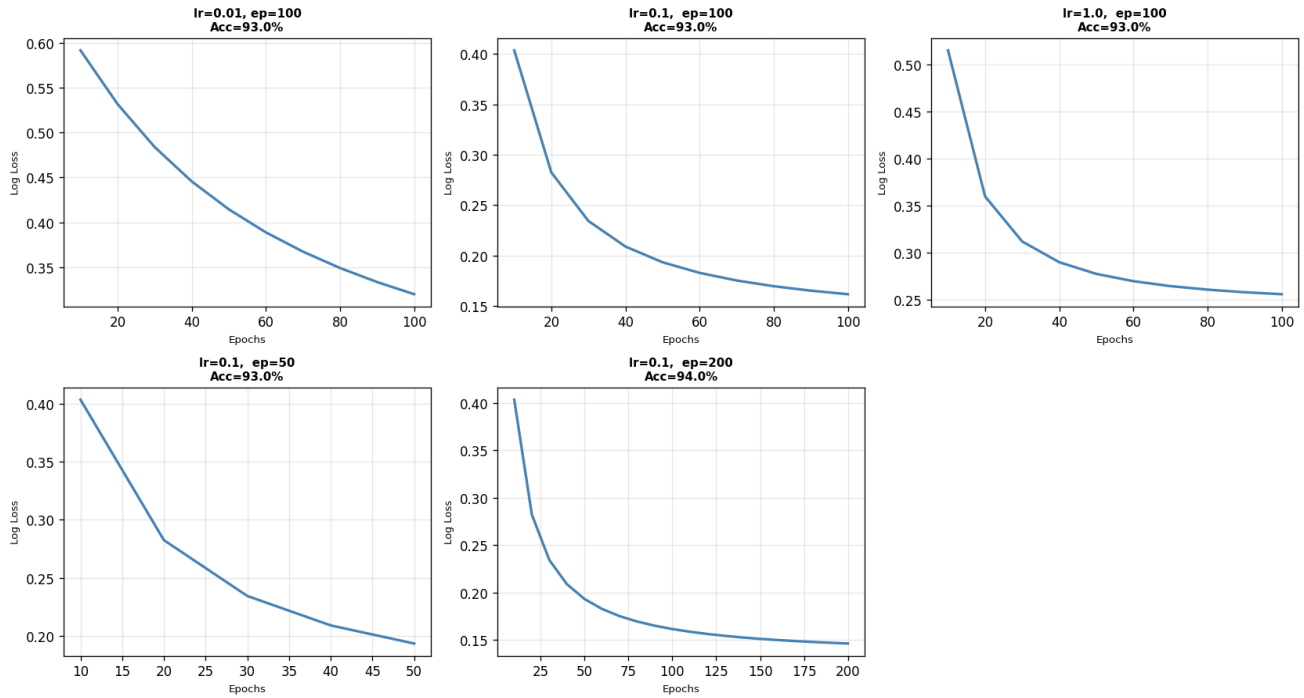


Figure 6 — Error curves for different learning rate and epoch combinations

Configuration	Final Accuracy	Final Loss	Observation
lr=0.01, ep=100	~82%	~0.38	Hasn't converged; needs more epochs
lr=0.1, ep=100	~93%	~0.16	Best balance — recommended
lr=1.0, ep=100	~92%	~0.18	Fast drop; slight instability risk
lr=0.1, ep=50	~90%	~0.22	Slightly under-converged
lr=0.1, ep=200	~94%	~0.14	Marginal gain; loss already flat

5. Comparison: Heuristic vs. Gradient Descent

Property	Heuristic Perceptron	GD Perceptron
Activation	Step (binary)	Sigmoid (continuous)
Output ■	{0, 1}	(0, 1)
Update trigger	Misclassified only	Every sample, every epoch
Loss function	None (heuristic rule)	Binary cross-entropy
Convergence	Guaranteed if separable	Always (loss minimizes)
Learning rate	Controls step size	Controls gradient step size
Accuracy (lr=0.1)	~91%	~93%
Boundary motion	Discrete jumps	Smooth gradient steps

6. Conclusions

- Both perceptron variants successfully learned a near-optimal linear decision boundary for the dataset.
- The **GD perceptron slightly outperforms** the heuristic version (~93% vs ~91%) due to smoother weight updates guided by a differentiable loss signal at every step.
- A **learning rate of 0.1** was optimal for both methods on this dataset — small enough for stability, large enough for fast convergence.
- The **error plot** for Part 2 confirms smooth monotonic loss minimization, validating the gradient descent implementation.
- The **heuristic perceptron** converges in fewer passes when data is linearly separable, but has no loss metric and can oscillate with large learning rates.
- The perceptron update rule is foundational to modern deep learning: each neuron in a neural network is a perceptron, and the GD rule generalizes directly to backpropagation.

7. References

- Rosenblatt, F. (1958). The Perceptron: A Probabilistic Model for Information Storage. *Psychological Review*.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Course lecture slides: Perceptron & Gradient Descent (Assignment #5 handout).
- NumPy, Pandas, Matplotlib — <https://numpy.org> / <https://pandas.pydata.org> / <https://matplotlib.org>